In [2]:
```
import pandas as pd
```

Pandas

Pandas is a powerful, open-source data analysis and manipulation library built on top of NumPy. It provides:

DataFrame: 2D table-like structure (like Excel spreadsheet)

Series: 1D labeled array (single column of data)

Tools for reading/writing data from various formats

Data cleaning, transformation, and analysis capabilities

Series

In [3]:
```
s1 = pd.Series(["a","b","c","d"])
```

In [4]:
```
s1
```

Out[4]:
```
0    a
1    b
2    c
3    d
dtype: object
```

In [ ]:
```

```

In [5]:
```
##Syntax: s2 = pd.Series([values],index=[indices])

s2 = pd.Series([10,20,30,40],index=["a","b","c","d"])
s2
```

Out[5]:
```
a    10
b    20
c    30
d    40
dtype: int64
```

From a Dict

In [6]:
```
#create a series using dictionary consisting name and age of the users

dictSeries = pd.Series({'Waleed':'20', 'Ahmed':17,'Ali':'25'})
print(dictSeries)
```

```
Waleed    20
Ahmed     17
Ali       25
dtype: object
```

Retreival/Selection of Series

In [7]:
```
## Single Label

print(s1[1])
print(dictSeries['Waleed'])
print(s1[3])

## List of Labels
print("\nList of Labels")
print(s1[[1,2]])
print(s2[["a","c"]])


##Filtering Conditions

# using mask
mask = s2>10
print(s2[mask])

##or
##directly conditioning in print
print(s2[s2>10])
```

```
b
20
d

List of Labels
1    b
2    c
dtype: object
a    10
c    30
dtype: int64
b    20
c    30
d    40
dtype: int64
b    20
c    30
d    40
dtype: int64
```

Series Attributes

In [8]:
```
print(s1.values)
print(s1.index)
print(s1.dtype)
print(s1.size)
print(s1.shape)
print(s1.ndim)
```

```
['a' 'b' 'c' 'd']
RangeIndex(start=0, stop=4, step=1)
object
4
(4,)
1
```

Series Operations

In [19]:
```
s = pd.Series([1,2,3,4,5])

print(s+10)
print(s*s)

# Statistical operations
print("Mean:", s.mean())
print("Sum:", s.sum())
print("Median:", s.median())
print("Describe:\n",s.describe())
```

```
0    11
1    12
2    13
3    14
4    15
dtype: int64
0     1
1     4
2     9
3    16
4    25
dtype: int64
Mean: 3.0
Sum: 15
Median: 3.0
Describe:
 count    5.000000
mean     3.000000
std      1.581139
min      1.000000
25%      2.000000
50%      3.000000
75%      4.000000
max      5.000000
dtype: float64
```

DataFrame

From Dictionary

In [5]:
```
Dict = {
```

```
    'Name' : ['Waleed','Ali','Ahmed'],
    'Age' : [20,30,19],
    'Country' : ['PK','IN','US']
}

DF_Dict = pd.DataFrame(Dict)
print(DF_Dict)
print()
print(DF_Dict.values)
print()
print(DF_Dict.index)
```

```
    Name  Age Country
0  Waleed   20      PK
1     Ali   30      IN
2   Ahmed   19      US

[['Waleed' 20 'PK']
 ['Ali' 30 'IN']
 ['Ahmed' 19 'US']]

RangeIndex(start=0, stop=3, step=1)
```

From List of Lists

In [12]:
```
List = [
    ['Waleed',20,'Karachi',800000],
    ['Ahmed',40,'Multan',300000],
    ['Ali',13,'Lahore',550000]
]

cols = ['Name','Age','City','Salary'] ##list for setting the custom columns name

DF_List = pd.DataFrame(List,columns=cols)
print(DF_List)
print()


##We can also give custom Index names

idx = ['emp01','emp02','emp03']
DF_List = pd.DataFrame(List,index=idx,columns=cols)

print('With Custom Index Names')
print(DF_List)
```

```
    Name  Age     City  Salary
0  Waleed   20  Karachi  800000
1   Ahmed   40   Multan  300000
2     Ali   13   Lahore  550000

With Custom Index Names
         Name  Age     City  Salary
emp01  Waleed   20  Karachi  800000
emp02   Ahmed   40   Multan  300000
emp03     Ali   13   Lahore  550000
```

From CSV File

In [36]:
```
##ageGreater18 = DF_List["Age"]>18
# print(type(DF_List["Age"]))

mask = DF_List["Age"]>18
print(mask)
DF_List.loc[mask]
```

```
emp01     True
emp02     True
emp03    False
Name: Age, dtype: bool
```

Out[36]:

	Name	Age	City	Salary
emp01	Waleed	20	Karachi	800000
emp02	Ahmed	40	Multan	300000

In [20]:
```
csvDF= pd.read_csv("elections.csv")
```

In [21]:
```
csvDF
```

Out[21]:

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789
...	...	...	...	...	...	...
182	2024	Donald Trump	Republican	77303568	win	49.808629
183	2024	Kamala Harris	Democratic	75019230	loss	48.336772
184	2024	Jill Stein	Green	861155	loss	0.554864
185	2024	Robert Kennedy	Independent	756383	loss	0.487357
186	2024	Chase Oliver	Libertarian Party	650130	loss	0.418895

187 rows × 6 columns

Using Loc and iloc

In [127…

```
#Use iloc to select the first 10 rows and only the 'Candidate' and 'Party' columns.
csvDF.iloc[:10,[2,3]]

#Use loc to select all rows from 2020 to 2024 (inclusive) and show only the 'Year', 'Candidate', and 'Result' c
colsData = csvDF.loc[:,["Year","Candidate","Result"]]
colsData[(colsData["Year"]>2020) & (colsData["Year"]<=2024)]

#Use loc to find all Republican candidates who won elections
csvDF.loc[(csvDF["Party"]=="Republican") & (csvDF["Result"]=="win")]
csvDF.loc[(csvDF["Party"]=="Democratic") & (csvDF["%"] > 50.00) & (csvDF["Result"]=="loss")]

df= csvDF.copy()
df

df.loc[(df["Year"]<1900, 'Result')]="loss"
df["Result"]
```

Out[127…

```
0      loss
1      loss
2      loss
3      loss
4      loss
       ...
182     win
183    loss
184    loss
185    loss
186    loss
Name: Result, Length: 187, dtype: object
```

In [57]:

```
WinCandidates = (csvDF["Popular vote"]>30000) & (csvDF["Result"] == 'win')
# print(WinCandidates.loc['Year'])
# csvDF[WinCandidates,['Year']]
# print(csvDF[WinCandidates.loc["Year"]])

cols = csvDF.columns.values
print(cols)

print(csvDF.dtypes)

for i in cols:
    print(type(i))
```

```
['Year' 'Candidate' 'Party' 'Popular vote' 'Result' '%']
Year             int64
Candidate       object
Party           object
Popular vote     int64
Result          object
%              float64
dtype: object
<class 'str'>
<class 'str'>
<class 'str'>
<class 'str'>
<class 'str'>
<class 'str'>
```

In [74]:
```
# csvDF.loc[WinCandidates.head]
# print(WinCandidates)

# mask = csvDF.loc[csvDF['Year']>2023]
# print(mask)

mask = csvDF['Year']>2023
mask
```

Out[74]:
```
0      False
1      False
2      False
3      False
4      False
       ...
182     True
183     True
184     True
185     True
186     True
Name: Year, Length: 187, dtype: bool
```

In [103…
```
# csvDF[['Year','Candidate']]
mask = csvDF[csvDF['Year']>2000][['Year','Candidate']]
print(mask)
```

```
     Year         Candidate
156  2004        David Cobb
157  2004    George W. Bush
158  2004        John Kerry
159  2004  Michael Badnarik
160  2004  Michael Peroutka
161  2004      Ralph Nader
162  2008     Barack Obama
163  2008         Bob Barr
164  2008    Chuck Baldwin
165  2008  Cynthia McKinney
166  2008      John McCain
167  2008      Ralph Nader
168  2012     Barack Obama
169  2012     Gary Johnson
170  2012       Jill Stein
171  2012      Mitt Romney
172  2016    Darrell Castle
173  2016      Donald Trump
174  2016     Evan McMullin
175  2016      Gary Johnson
176  2016   Hillary Clinton
177  2016       Jill Stein
178  2020      Joseph Biden
179  2020      Donald Trump
180  2020      Jo Jorgensen
181  2020    Howard Hawkins
182  2024      Donald Trump
183  2024     Kamala Harris
184  2024        Jill Stein
185  2024    Robert Kennedy
186  2024      Chase Oliver
```

In [115…
```
DC = csvDF.loc[csvDF['Party']=="Democratic",'Candidate']
DC
```

Out[115…
```
2              Andrew Jackson
4              Andrew Jackson
8            Martin Van Buren
10           Martin Van Buren
13                 James Polk
14                 Lewis Cass
17             Franklin Pierce
20              James Buchanan
28         George B. McClellan
29             Horatio Seymour
34            Samuel J. Tilden
37     Winfield Scott Hancock
39           Grover Cleveland
45           Grover Cleveland
47           Grover Cleveland
52     William Jennings Bryan
55     William Jennings Bryan
57            Alton B. Parker
64     William Jennings Bryan
70            Woodrow Wilson
74            Woodrow Wilson
77              James M. Cox
81             John W. Davis
83                  Al Smith
86         Franklin Roosevelt
91         Franklin Roosevelt
94         Franklin Roosevelt
97         Franklin Roosevelt
100              Harry Truman
105           Adlai Stevenson
108           Adlai Stevenson
111              John Kennedy
114            Lyndon Johnson
116           Hubert Humphrey
118           George McGovern
123              Jimmy Carter
129              Jimmy Carter
134            Walter Mondale
137           Michael Dukakis
140              Bill Clinton
144              Bill Clinton
151                   Al Gore
158                John Kerry
162              Barack Obama
168              Barack Obama
176           Hillary Clinton
178              Joseph Biden
183             Kamala Harris
Name: Candidate, dtype: object
```

In [124…
```
RW = csvDF.loc[(csvDF["Party"]=="Republican") & (csvDF["Result"] == "win") & (csvDF["Year"]>2000)]
RW
```

Out[124…

	Year	Candidate	Party	Popular vote	Result	%
157	2004	George W. Bush	Republican	62040610	win	50.771824
173	2016	Donald Trump	Republican	62984828	win	46.407862
182	2024	Donald Trump	Republican	77303568	win	49.808629

In [129…
```
Lost50 = csvDF.loc[(csvDF["Result"]=="loss") & (csvDF["%"] > 50.00)]
Lost50
```

Out[129…

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
34	1876	Samuel J. Tilden	Democratic	4288546	loss	51.528376

In [135…
```
Margin2 = csvDF.loc[(csvDF["%"]>48) & (csvDF["%"]<52)]
Margin2
```

Out[135…

	Year	Candidate	Party	Popular vote	Result	%
12	1844	Henry Clay	Whig	1300004	loss	49.250523
13	1844	James Polk	Democratic	1339570	win	50.749477
17	1852	Franklin Pierce	Democratic	1605943	win	51.013168
33	1876	Rutherford Hayes	Republican	4034142	win	48.471624
34	1876	Samuel J. Tilden	Democratic	4288546	loss	51.528376
36	1880	James Garfield	Republican	4453337	win	48.369234
37	1880	Winfield Scott Hancock	Democratic	4444976	loss	48.278422
39	1884	Grover Cleveland	Democratic	4914482	win	48.884933
40	1884	James G. Blaine	Republican	4856905	loss	48.312208
45	1888	Grover Cleveland	Democratic	5534488	loss	48.656799
53	1896	William McKinley	Republican	7112138	win	51.213817
74	1916	Woodrow Wilson	Democratic	9126868	win	49.367987
100	1948	Harry Truman	Democratic	24179347	win	49.601536
111	1960	John Kennedy	Democratic	34220984	win	50.082561
112	1960	Richard Nixon	Republican	34108157	loss	49.917439
122	1976	Gerald Ford	Republican	39148634	loss	48.199499
123	1976	Jimmy Carter	Democratic	40831881	win	50.271900
131	1980	Ronald Reagan	Republican	43903230	win	50.897944
144	1996	Bill Clinton	Democratic	47400125	win	49.296938
151	2000	Al Gore	Democratic	50999897	loss	48.491813
157	2004	George W. Bush	Republican	62040610	win	50.771824
158	2004	John Kerry	Democratic	59028444	loss	48.306775
168	2012	Barack Obama	Democratic	65915795	win	51.258484
176	2016	Hillary Clinton	Democratic	65853514	loss	48.521539
178	2020	Joseph Biden	Democratic	81268924	win	51.311515
182	2024	Donald Trump	Republican	77303568	win	49.808629
183	2024	Kamala Harris	Democratic	75019230	loss	48.336772

In [139…

```
ThirdParty = csvDF.loc[~((csvDF["Party"]=="Democratic") | (csvDF["Party"]=="Republican"))]
ThirdParty
```

Out[139…

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
5	1832	Henry Clay	National Republican	484205	loss	37.603628
6	1832	William Wirt	Anti-Masonic	100715	loss	7.821583
...	...	...	...	...	...	...
180	2020	Jo Jorgensen	Libertarian	1865724	loss	1.177979
181	2020	Howard Hawkins	Green	405035	loss	0.255731
184	2024	Jill Stein	Green	861155	loss	0.554864
185	2024	Robert Kennedy	Independent	756383	loss	0.487357
186	2024	Chase Oliver	Libertarian Party	650130	loss	0.418895

97 rows × 6 columns

Group By

Group By is one of the most powerful operations in pandas. It allows you to:

Split data into groups based on criteria

Apply functions to each group independently

Combine the results back together

In [140…
```
elect = csvDF.copy()
# groupedParty = elect.groupby("Party")
# groupedParty.size()

## Basic Aggregation

#### Average vote percentage by party
party_group = elect.groupby("Party")["%"].mean()
 ## or party_group = elect.groupby("Party")["%"].agg(['mean'])
party_group

## Multiple Aggregation

#### Multiple Aggregation for each party
partyDetails = elect.groupby("Party")["%"].agg(['mean','max','min','count'])
partyDetails

##Grouping by Multiple Columns

##### Group by both Party and Result
MultiGroup = elect.groupby(["Party","Result"])["%"].mean()
MultiGroup

win_loss_count = elect.groupby(['Party', 'Result']).size()
print(win_loss_count.head(5))
```

```
Party                 Result
American              loss      2
American Independent  loss      3
Anti-Masonic          loss      1
Anti-Monopoly         loss      1
Citizens              loss      1
dtype: int64
```

In [141…
```
partyAvg = elect.groupby('Party')['%'].mean().count()
partyAvg
```

Out[141… `np.int64(37)`

In [315…
```
#Count how many times each party appears in the dataset
partyTimes = elect.groupby('Party').size()
partyTimes

#Calculate the total popular votes received by each party across all years (sum)
totalPopular = elect.groupby('Party')["Popular vote"].sum()
totalPopular.head()

#For each election year, find how many candidates participated (count or size)
candCounts = elect.groupby("Year")["Candidate"].count()
candCounts.head()

#Show the maximum vote percentage achieved by each party
maxVotes = elect.groupby("Party")["%"].max()
maxVotes.head()
```

Out[315…
```
Party
American               21.554001
American Independent   13.571218
Anti-Masonic            7.821583
Anti-Monopoly           1.335838
Citizens                0.270182
Name: %, dtype: float64
```

Filtering by Group By

In [229…
```
##Keep parties whose candidates average more than 20% of the vote
high_avg_parties = elect.groupby("Party").filter(lambda group: group["%"].mean()>20)
print(high_avg_parties["Party"].unique())

print()
## Keep parties that have achieved at least 40% in any election
max40 = elect.groupby("Party").filter(lambda group: group["%"].max()>40)
print(max40["Party"].unique())

print()
## Keep parties that have candidates in elections from 1970 onwards
OldParties = elect.groupby("Party").filter(lambda group: group["Year"].max()>1970)
print(OldParties)
```

```
['Democratic-Republican' 'Democratic' 'National Republican' 'Whig'
 'Republican' 'Northern Democratic' 'National Union' 'Liberal Republican']

['Democratic-Republican' 'Democratic' 'National Republican' 'Whig'
 'Republican' 'National Union' 'Liberal Republican']

     Year         Candidate              Party  Popular vote Result  \
2    1828    Andrew Jackson         Democratic        642806    win
4    1832    Andrew Jackson         Democratic        702735    win
8    1836  Martin Van Buren         Democratic        763291    win
10   1840  Martin Van Buren         Democratic       1128854   loss
13   1844        James Polk         Democratic       1339570    win
..    ...               ...                ...           ...    ...
182  2024      Donald Trump         Republican      77303568    win
183  2024     Kamala Harris         Democratic      75019230   loss
184  2024        Jill Stein              Green        861155   loss
185  2024    Robert Kennedy        Independent        756383   loss
186  2024      Chase Oliver  Libertarian Party        650130   loss

             %    Win
2    56.203927   True
4    54.574789   True
8    52.272472   True
10   46.948787  False
13   50.749477   True
..         ...    ...
182  49.808629   True
183  48.336772  False
184   0.554864  False
185   0.487357  False
186   0.418895  False

[135 rows x 7 columns]
```

In [316…

```
#### GROUP BY QUESTIONS #######

##Find the average vote percentage for each political party.
elect.groupby("Party")["%"].mean()

##Calculate the win rate (percentage of wins) for each major party (Democratic and Republican)
elect['Win'] = elect["Result"]=="win"
Parties = elect.groupby('Party')['Win'].mean()
Parties.loc[["Democratic","Republican"]]

## Group by year and find which year had the most candidates.
mostCands = elect.groupby(["Year"])["Candidate"].count()
year_with_most = mostCands.idxmax() ### Find the year with most candidates
most_cands_count = mostCands.max()  ###calculate the max of the count of the candidates
print("Year: ",year_with_most , "|| Candidates: ",most_cands_count)

##For each party, calculate the minimum, maximum, and average popular vote received.
partyDetails = elect.groupby("Party")["Popular vote"].agg(["min","max","mean"])
print(partyDetails.head(10))


## Find the candidate with the highest vote percentage in each party.
print()
highestVotes = elect.groupby("Candidate")["%"].max()
print(highestVotes.idxmax())
print(highestVotes.max())
```

```
Year:  1996 || Candidates:  7
                          min       max          mean
Party
American               158271    873053  5.156620e+05
American Independent   170274   9901118  3.724087e+06
Anti-Masonic           100715    100715  1.007150e+05
Anti-Monopoly          134294    134294  1.342940e+05
Citizens               233052    233052  2.330520e+05
Communist              103307    103307  1.033070e+05
Constitution           143630    203091  1.821570e+05
Constitutional Union   590901    590901  5.909010e+05
Democratic             642806  81268924  2.386054e+07
Democratic-Republican  113142    151271  1.322065e+05

Lyndon Johnson
61.34470329
```

In [317…

```
elect
```

Out[317…

	Year	Candidate	Party	Popular vote	Result	%	Win
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122	False
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878	True
2	1828	Andrew Jackson	Democratic	642806	win	56.203927	True
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073	False
4	1832	Andrew Jackson	Democratic	702735	win	54.574789	True
...	...	...	...	...	...	...	...
182	2024	Donald Trump	Republican	77303568	win	49.808629	True
183	2024	Kamala Harris	Democratic	75019230	loss	48.336772	False
184	2024	Jill Stein	Green	861155	loss	0.554864	False
185	2024	Robert Kennedy	Independent	756383	loss	0.487357	False
186	2024	Chase Oliver	Libertarian Party	650130	loss	0.418895	False

187 rows × 7 columns

Pivot Table

pd.pivot_table(data, values=None, index=None, columns=None, aggfunc='mean', fill_value=None

Parameter Description Example

`data` DataFrame to use elect

`values` Column(s) to aggregate '%', 'Popular vote'

`index` Column(s) for row labels 'Year', 'Party'

`columns` Column(s) for column labels 'Result', 'Party'

`aggfunc` Aggregation function 'mean', 'sum', 'count'

`fill_value` Value to replace NaN 0, -1

In [318…

```
# Basic pivot: Average vote % by Party and Result

p1 = elect.pivot_table(
    index = "Year",
    columns = "Result",
    values = "%",
    aggfunc = "mean"
)

p1.head(10)
```

Out[318…

Result	loss	win
Year		
1824	57.210122	42.789878
1828	43.796073	56.203927
1832	22.712605	54.574789
1836	23.863764	52.272472
1840	46.948787	53.051213
1844	49.250523	50.749477
1848	26.345352	47.309296
1852	24.493416	51.013168
1856	27.346960	45.306080
1860	20.100197	39.699408

In [319…

```
# Count candidates by Party and Result

pivot2 = elect.pivot_table(
    index="Party",
    columns = "Result",
    values = "Candidate",
    aggfunc = "count",
    fill_value = "0"
```

```
)

pivot2.head()
```

Out[319…

Result	loss	win
Party		
American	2	0
American Independent	3	0
Anti-Masonic	1	0
Anti-Monopoly	1	0
Citizens	1	0

In [326…

```
# Multiple statistics for Popular vote by Party

p3 = elect.pivot_table(
    index = "Party",
    aggfunc = ['mean','max','min'],
    values = "Popular vote",
    fill_value = "0"
)

p3.head()
```

Out[326…

	mean	max	min
	Popular vote	Popular vote	Popular vote
Party			
American	5.156620e+05	873053	158271
American Independent	3.724087e+06	9901118	170274
Anti-Masonic	1.007150e+05	100715	100715
Anti-Monopoly	1.342940e+05	134294	134294
Citizens	2.330520e+05	233052	233052

Joins

In [329…

```
# Students table
students = pd.DataFrame({
    'student_id': [101, 102, 103, 104],
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'age': [16, 17, 16, 18]
})

# Scores table
scores = pd.DataFrame({
    'student_id': [101, 102, 103, 105],
    'math_score': [85, 92, 78, 88],
    'english_score': [90, 85, 92, 79]
})

print(students)
print(scores)
```

```
   student_id     name  age
0         101    Alice   16
1         102      Bob   17
2         103  Charlie   16
3         104    David   18
   student_id  math_score  english_score
0         101          85             90
1         102          92             85
2         103          78             92
3         105          88             79
```

In [335…

```
merged = pd.merge(left=students, right=scores)
merged

# or

merged = pd.merge(left=students, right=scores,left_on="student_id",right_on="student_id")
merged
```

Out[335…

	student_id	name	age	math_score	english_score
0	101	Alice	16	85	90
1	102	Bob	17	92	85
2	103	Charlie	16	78	92

In [337…

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_17020\997823177.py in ?()
----> 1 merged.sort_values("count", ascending=True)

D:\Programs\Anaconda\Lib\site-packages\pandas\core\frame.py in ?(self, by, axis, ascending, inplace, kind, na_po
sition, ignore_index, key)
   7185             )
   7186         elif len(by):
   7187             # len(by) == 1
   7188
-> 7189             k = self._get_label_or_level_values(by[0], axis=axis)
   7190
   7191             # need to rewrap column in Series to apply key function
   7192             if key is not None:

D:\Programs\Anaconda\Lib\site-packages\pandas\core\generic.py in ?(self, key, axis)
   1907             values = self.xs(key, axis=other_axes[0])._values
   1908         elif self._is_level_reference(key, axis=axis):
   1909             values = self.axes[axis].get_level_values(key)._values
   1910         else:
-> 1911             raise KeyError(key)
   1912
   1913         # Check for duplicates
   1914         if values.ndim > 1:

KeyError: 'count'
```

In [325…

```
pwd
```

Out[325… 'C:\\Users\\Admin'

In [322…

```
std = pd.read_csv("students_data.csv")
```

In [323…

```
std
```

Out[323…

	student_id	name	age	gender	grade	math_score	english_score	science_score	enrolled_date	remarks
0	100	Zunaira AHMED	16.0	female	11	75.0	NaN	66	10/06/2022	excellent
1	101	shams abid	16.0	Male	10th	74.0	95	94	10/06/2022	GOOD
2	102	hamza SHAH	NaN	MALE	10	NaN	missing	69	06/12/2022	needs improvement
3	103	Zunaira AHMED	16.0	FEMALE	10	NaN	missing	62	10/06/2022	average
4	104	SARA ALI	16.0	male	11	NaN	96	64	10/06/2022	GOOD
5	105	Maha khan	16.0	Female	10	NaN	NaN	83	06/12/2022	needs improvement
6	106	aliza nasir	17.0	female	11	64.0	NaN	75	06/12/2022	Good
7	107	SARA ALI	17.0	female	12	NaN	63	62	11/06/2022	excellent
8	108	Maha khan	16.0	Female	12	80.0	missing	89	06/12/2022	poor
9	109	nida Zubair	17.0	female	12	NaN	missing	97	10/06/2022	needs improvement
10	110	ahmed noor	17.0	male	11th	65.0	67	100	06/12/2022	excellent
11	111	aliza nasir	16.0	FEMALE	11th	NaN	missing	95	10/06/2022	average
12	112	SAIRA nasir	17.0	female	11	NaN	87	89	10/06/2022	poor
13	113	SAIRA nasir	17.0	female	10	NaN	NaN	98	06/12/2022	Average
14	114	aliza nasir	17.0	male	12	NaN	91	67	11/06/2022	poor
15	115	ahmed noor	16.0	Female	12	65.0	91	94	06/12/2022	Average
16	116	hamza SHAH	NaN	Female	11	NaN	NaN	72	11/06/2022	excellent
17	117	aliza nasir	18.0	male	11th	67.0	74	81	10/06/2022	GOOD
18	118	aliza nasir	17.0	Male	10	100.0	74	62	11/06/2022	average
19	119	SAIRA nasir	17.0	MALE	11th	73.0	NaN	90	06/12/2022	Good
20	120	SARA ALI	17.0	Male	10	NaN	missing	89	10/06/2022	average
21	121	shams abid	18.0	female	11	66.0	72	94	10/06/2022	Average
22	122	SARA ALI	17.0	MALE	11th	75.0	NaN	66	10/06/2022	good student
23	123	Zunaira AHMED	17.0	Female	11th	NaN	missing	63	06/12/2022	excellent
24	124	shams abid	18.0	MALE	11th	NaN	NaN	91	06/12/2022	GOOD
25	125	Maha khan	17.0	Male	12	94.0	80	63	10/06/2022	Average
26	126	shams abid	NaN	Male	10th	64.0	missing	67	06/12/2022	good student
27	127	aliza nasir	16.0	FEMALE	12	NaN	76	80	06/12/2022	poor
28	128	SARA ALI	17.0	female	11	NaN	64	89	10/06/2022	average
29	129	SAIRA nasir	17.0	FEMALE	11	NaN	64	83	11/06/2022	Average
30	129	SAIRA nasir	17.0	FEMALE	11	NaN	64	83	11/06/2022	Average

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: